

GPNEC: A Metal State Automaton on Apple Silicon

Verified Lattice Boltzmann Parity, Dual-Geometry Routing under Matched Policies, and Honest Performance Framing

Digital Defiance

August 2026

Abstract

We present **GPNEC**, an open Apple Silicon engine for hand-crafted state automata $X_{t+1} = \Phi(X_t, W, \theta)$ using Metal compute kernels and, where topology is a dense linear map, MPSGraph. This report focuses on what we can verify with reproducible CLI gates on an Apple M4 Max.

For D2Q9 Lattice Boltzmann (BGK), a single-thread CPU reference that mirrors the Metal collide-stream kernels matches GPU state to relative $L_2 \sim 3 \times 10^{-7}$. We report Metal throughput in MLups and estimate effective memory traffic, and we explicitly treat the single-thread CPU mirror as an *accuracy gold standard*, not as a competitive CFD baseline. Headline “speedup vs one CPU thread” figures are therefore secondary and carefully scoped.

For dual-geometry greedy routing, Metal and CPU hop updates agree with zero field mismatches under lockstep comparison. Under a **symmetric** post-crash protocol—identical silent-retry and respawn on both metrics—Poincaré cumulative deliveries remain higher than Euclidean on our dual embedding (about $1.8\times$ in the primary table). We also report the Route app’s asymmetric “sandbox” protocol for completeness, and we state plainly that it handicaps Euclidean traffic and must not be used as an embedding-only resilience claim.

Source, `gp nec verify`, and paired `route-verify --policy both` gates are released with the measurements archived under `papers/gpnec/`.

Contents

1	Introduction	3
2	System architecture	3
2.1	Automaton API	3
2.2	Where MPSGraph ends and Metal begins	3
3	Lattice Boltzmann	3
3.1	Model, Mach number, and boundaries	3
3.2	CPU reference (accuracy gold, not a CFD champion)	4
3.3	Accuracy protocol	4
3.4	Throughput and bandwidth estimate	4
4	Dual-geometry routing	4
4.1	Dual embedding	4
4.2	Greedy hop and float32 Poincaré clamping	5
4.3	Metal $\equiv$ CPU lockstep	5
4.4	Crash protocols: symmetric (primary) vs sandbox (narrative)	5
5	Experimental setup	5

6	Results	5
6.1	LBM accuracy	5
6.2	LBM throughput (scoped)	5
6.3	Routing: lockstep and paired crash policies	6
7	Limitations and non-claims	6
8	Reproducibility	7
9	Conclusion	7

1 Introduction

GPNEC evaluates local update rules on Apple Silicon without an ML training stack: no Core ML, MLX, or PyTorch in the core loop. Geometry lives in W , transition physics or routing policy in a Metal kernel Φ , and the host API stays stable across adapters.

This revision addresses methodological gaps that would (correctly) draw reviewer criticism in a systems or computational-physics venue:

1. Routing comparisons now lead with a **symmetric** crash control (identical retry/respawn). The UI “freeze Euclidean” protocol is demoted to a demo narrative.
2. LBM performance is framed primarily as Metal MLups and bandwidth utilization against a kernel-faithful CPU accuracy reference—not as a $2000\times$ claim against a competitive multi-core CFD code.
3. We separate MPSGraph topology paths from custom Metal LBM/route kernels, document Poincaré float32 clamping, and state LBM Mach / boundary assumptions for the verified bench path.

2 System architecture

2.1 Automaton API

$$X_{t+1} = \Phi(X_t, W, \theta). \quad (1)$$

State lives in double-buffered `MTLBuffers` on unified memory.

2.2 Where MPSGraph ends and Metal begins

The marketing phrase “Metal/MPSGraph automaton” covers two execution paths that must not be conflated:

Path	Mechanism
Topology matmul	<code>MPSGraph</code> batched $Y = WX$ then custom Φ
Collide+stream	Two custom Metal kernels (<code>lbm_bgk_collide</code> , <code>lbm_stream_bounce</code> in <code>PhiLBM.metal</code>); M
Dual greedy route	Custom Metal kernel <code>dual_greedy_route_step</code>

In short: MPSGraph manages dense linear topology where that is the natural W ; the headline LBM and dual-router results in this paper are custom Metal kernels.

3 Lattice Boltzmann

3.1 Model, Mach number, and boundaries

We use D2Q9 BGK Succi [2001], Krüger et al. [2017] with lattice sound speed $c_s = 1/\sqrt{3}$. Default demo inlet $u_x = 0.08$ gives

$$\text{Ma} \approx \frac{u_x}{c_s} \approx 0.14, \quad (2)$$

comfortably in the low-Mach regime where BGK is commonly applied. Relaxation time $\tau = 0.56$ (clamped ≥ 0.51 in kernels).

The stream kernel applies: periodic boundaries in y ; approximate bounce on open x faces; mid-grid bounce-back from solid masks; and a left-column velocity inlet that overwrites populations with equilibrium at $(\rho, u) = (1, (u_x, 0))$. Dye (channel 9) is advected upwind. The **accuracy/bench path** used in tables below disables solids and uses matched inlet settings so Metal and CPU see identical IC/BC; the Fluid UI adds cylinder solids for visualization without changing the verify contract.

3.2 CPU reference (accuracy gold, not a CFD champion)

`cpu-d2q9` is a single-thread Swift implementation that mirrors `PhiLBM.metal` channel layout and arithmetic. Its job is **algorithmic parity** for relative L_2 tests. It is intentionally not NEON-tuned multipole CFD. Optional `cpu-mt` parallelizes over rows; on this dense stencil it is typically *slower* than single-thread (memory/contention), so we report it only to discourage using a weak multi-thread straw man as a “fixed” baseline.

We therefore **do not** treat “Metal / single-thread CPU” ratios as the primary systems claim. Primary claims are: (i) Metal $\equiv$ CPU state parity, and (ii) achieved Metal MLups with an order-of-magnitude bandwidth estimate.

3.3 Accuracy protocol

`gpnc verify-lbm` compares full 10-channel state after N steps. Gate: relative $L_2 \leq 10^{-4}$ (observed $\sim 10^{-7}$), finite values, and rest-equilibrium drift on CPU.

3.4 Throughput and bandwidth estimate

Let $N_c = W \cdot H$ cells and $C = 10$ float32 channels (D2Q9+dye). One collide+stream step performs two full read/write passes over the state in our ping-pong implementation, i.e. about

$$T \approx 4 \cdot N_c \cdot C \cdot 4 \text{ bytes} = 16 N_c C \text{ bytes} \quad (3)$$

of explicit buffer traffic (ignoring solids texture and incidental reads). For 256^2 this is ≈ 10.5 MiB/step. At $69.9 \mu\text{s}/\text{step}$ (Table 2),

$$B_{\text{eff}} \approx \frac{10.5 \times 2^{20}}{69.9 \times 10^{-6}} \approx 150 \text{ GB/s}. \quad (4)$$

Published M4 Max unified memory bandwidth is on the order of 500+ GB/s; ~ 150 GB/s is therefore only a modest fraction of peak. At these lattice sizes the working set (~ 2.5 MiB) fits in caches, so the step time is plausibly limited by **kernel launch / command-buffer overhead and latency** rather than sustained DRAM bandwidth. Larger lattices, fused collide-stream, or persistent command buffers would be needed before claiming a bandwidth roofline result. We report this estimate so readers do not over-interpret absolute MLups.

4 Dual-geometry routing

4.1 Dual embedding

Sample Poincaré-disk points; build symmetrized K -NN edges in Poincaré distance; embed the *same* undirected graph in the plane via landmark MDS; select a crash set of size C along a vertical mid-strip of the Euclidean layout (Route app backbone cut), informed by sampled betweenness.

4.2 Greedy hop and float32 Poincaré clamping

One Metal thread per packet forwards to the live neighbor minimizing distance to the destination. Poincaré distance matches the CPU reference:

$$\text{if } \|a\| \geq 0.999, \quad a \leftarrow a \cdot 0.999 / \|a\| \quad (\text{likewise for } b), \quad (5)$$

$$d = \log(\alpha + \sqrt{\max(\alpha^2 - 1, 0)}), \quad \alpha = 1 + \frac{2\|a - b\|^2}{\max(1 - \|a\|^2, 10^{-6}) \max(1 - \|b\|^2, 10^{-6})}. \quad (6)$$

Clamping keeps points strictly inside the open disk so float32 denominators and acosh-equivalent logs remain finite near the boundary. This is a reproducibility detail, not a claim of exact hyperbolic isometry under all float32 roundings.

4.3 Metal≡CPU lockstep

With `stuckRetryHops=0`, Metal and CPU steppers are compared field-wise each tick. Zero mismatches is the correctness gate for the hop kernel.

4.4 Crash protocols: symmetric (primary) vs sandbox (narrative)

Symmetric (publication control). After the crash, *both* metrics keep `stuckRetryHops=40` and respawn finished slots. Any delivery gap is attributable to geometry and greedy progress on the damaged graph under matched mechanics.

Sandbox (UI narrative only). Matches the Route app: Euclidean sets `stuckRetryHops=0` and stops respawning; Poincaré keeps `retry+respawn`. This **handicaps** Euclidean traffic by construction. It is useful for a visual demo and is reported for fidelity to the shipped UI, but it is **not** evidence that the embedding alone explains large delivery multiples.

`gp nec verify-route` requires the symmetric gate for `verified: true`; sandbox results are printed alongside. `route-verify --policy both` prints both reports.

5 Experimental setup

Host: Apple M4 Max, macOS, Swift 6 / Metal. Transcripts: `papers/gpnec/verify-all.txt`, `bench.txt`, `route-verify-paired.txt`.

Primary route table: $N = 800$, $K = 10$, $C = 250$, 512 packets, 80/120 pre/post ticks, seed 42, 64 betweenness samples.

6 Results

6.1 LBM accuracy

Table 1: Metal vs CPU D2Q9 (64×64 , 32 steps, $\tau = 0.56$). Gate: relative $L_2 \leq 10^{-4}$.

Rel. L_2	Max $ \Delta $	Mass rel. Δ	Rest drift	Gate
3.28×10^{-7}	4.77×10^{-7}	$\sim 10^{-6}$	0	pass

6.2 LBM throughput (scoped)

`cpu-mt` is slower than single-thread here; we include it to show that naive multi-threading does not automatically produce a stronger baseline.

Table 2: LBM step time and MLups on M4 Max (warmup 64, timed 50). CPU columns are the kernel-faithful accuracy reference and a row-parallel variant—not tuned CFD baselines. Metal/CPU ratios are informational only.

Size	Metal μ s	Metal MLups	CPU μ s	CPU-mt μ s	vs CPU	B_{eff} est.
32^2	23.4	43.8	2357	11280	$101\times$	—
64^2	24.4	167.8	9633	47752	$395\times$	—
128^2	34.6	472.9	38013	195377	$1097\times$	—
256^2	69.9	937.3	155834	790508	$2229\times$	~ 150 GB/s

6.3 Routing: lockstep and paired crash policies

Table 3: `gpnec verify-route` checks (M4 Max, $N = 800$, seed 42).

Check	Outcome
Embedding invariants	pass
Metal $\equiv$ CPU hops (8×1024)	0 / 0 mismatches
Symmetric crash gate (publication)	pass
Sandbox crash (UI narrative; reported)	pass

Table 4: Paired crash comparison under identical seeds ($N = 800$, $K = 10$, $C = 250$, 512 packets, seed 42). Symmetric = matched retry+respawn. Sandbox = Euclidean freeze (handicapped).

Policy	Metric	Post delivered	In-flight	Hyp/Euc
Symmetric	Euclidean	544	512	1.78
Symmetric	Poincaré	966	512	1.78
Sandbox	Euclidean	125	387	7.65
Sandbox	Poincaré	956	512	7.65

Under **symmetric** mechanics, Poincaré still out-delivers Euclidean by about $1.8\times$ on this embedding and crash set—a meaningful but far more modest gap than the sandbox’s $7.7\times$, which largely reflects disabling Euclidean retry/respawn. Pre-crash, both metrics deliver substantial traffic (Euclidean 888, Poincaré 3092 cumulative under respawn), so the post-crash comparison is not an artifact of a dead pre-crash Euclidean mesh.

7 Limitations and non-claims

- **Routing generality.** Results are for one dual-embedding generator, one crash geometry, and listed seeds. We do not claim hyperbolic greedy routing dominates Euclidean greedy routing on arbitrary graphs.
- **Sandbox $\neq$ science claim.** Large sandbox multiples are expected under handicapped Euclidean policy; use symmetric numbers for embedding comparisons.
- **CPU performance.** Single-thread (and naive mt) CPU figures establish parity and rough scale; they are not a substitute for NEON/Accelerate or CUDA baselines on appropriate hardware.
- **Roofline.** The ~ 150 GB/s estimate at 256^2 is a back-of-envelope traffic model, not a full roofline with measured counters. Small lattices are likely launch/latency limited.

- **Cross-vendor.** No CUDA/OpenFOAM same-device study on this host.
- **Float32 hyperbolic math.** Clamping avoids NaNs near the disk boundary; it does not eliminate all geometric distortion from single-precision evaluation.

8 Reproducibility

```
swift test
swift run gpnc verify
swift run gpnc route-verify --n 800 --k 10 --crash 250 \
  --packets 512 --pre 80 --post 120 --seed 42 \
  --betweenness-samples 64 --policy both
swift run gpnc bench --backends metal,cpu,cpu-mt \
  --sizes 32,64,128,256 --steps 50 --warmup 64
```

9 Conclusion

GPNEC’s durable contributions are reproducible Metal $\equiv$ CPU gates for LBM and greedy routing, a fair symmetric crash control under which Poincaré still leads on our dual embedding, and performance reporting that separates accuracy references from competitive baselines. The asymmetric sandbox remains a UI demonstration, not a substitute for matched-policy evidence.

Availability

<https://github.com/Digital-Defiance/gpnc>

References

- Timm Krüger, Halim Kusumaatmaja, Alexandr Kuzmin, Orest Shardt, Goncalo Silva, and Erlend Magnus Vigen. *The Lattice Boltzmann Method: Principles and Practice*. Springer, 2017.
- Sauro Succi. *The Lattice Boltzmann Equation for Fluid Dynamics and Beyond*. Oxford University Press, 2001.